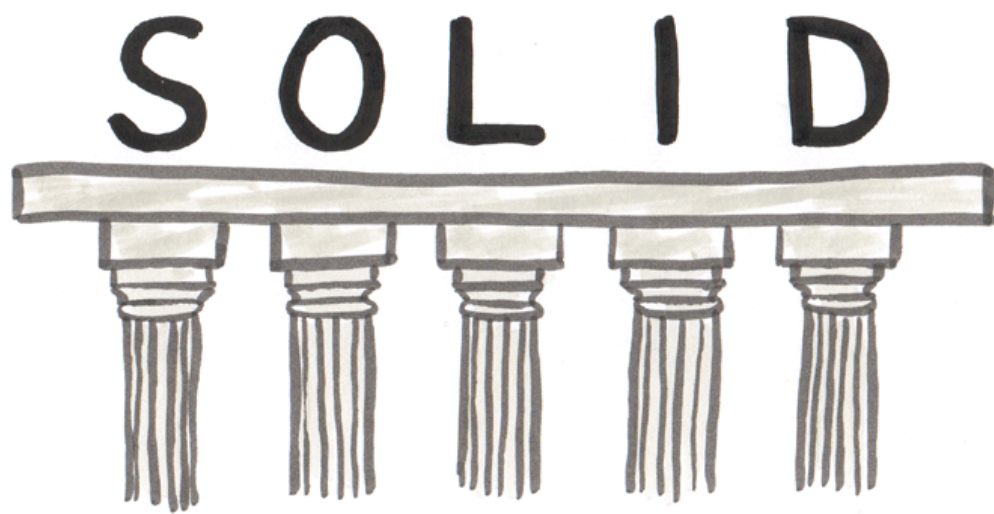


Pestanya 1

Refactoring



ÍNDICE

Ejemplo	3
Ejemplo 2	4
Ejemplo 3. Debugging.	6

Ejemplo

```
class Pedido:
    def __init__(self, productos, cliente):
        self.productos = productos
        self.cliente = cliente

    def calcular_total(self):
        total = 0
        for producto in self.productos:
            total += producto[1] * producto[2] # cantidad *
precio_unitario
        return total

    def aplicar_descuento(self, tipo_cliente):
        total = self.calcular_total()
        if tipo_cliente == "VIP":
            total *= 0.9 # 10% de descuento
        elif tipo_cliente == "frecuente":
            total *= 0.95 # 5% de descuento
        return total

    def generar_factura(self):
        factura = f"Factura para {self.cliente}:\n"
        for producto in self.productos:
            factura += f"{producto[0]} - {producto[1]} x
${producto[2]} = ${producto[1] * producto[2]}\n"
        factura += f"Total: ${self.calcular_total()}\n"
        return factura

# Uso
pedido = Pedido([("Laptop", 1, 1000), ("Ratón", 2, 25)], "Carlos")
print(pedido.generar_factura())
print("Total con descuento VIP:", pedido.aplicar_descuento("VIP"))
```

AYUDAS:

- Eliminar código duplicado
- Uso de estructuras de datos más claras
- Hacer el código más extensible

Ejemplo 2

```
class Calculator:
    # Suma dos números
    def add_two_numbers(self, num1, num2):
        result = num1 + num2
        print(f"The result of adding {num1} and {num2} is: {result}")
        return result

    # Suma tres números
    def add_three_numbers(self, num1, num2, num3):
        result = num1 + num2 + num3
        print(f"The result of adding {num1}, {num2} and {num3} is:
{result}")
        return result

    # Resta dos números
    def subtract_two_numbers(self, num1, num2):
        result = num1 - num2
        print(f"The result of subtracting {num2} from {num1} is: {result}")
        return result

    # Resta tres números
    def subtract_three_numbers(self, num1, num2, num3):
        result = num1 - num2 - num3
        print(f"The result of subtracting {num2} and {num3} from {num1} is:
{result}")
        return result

    # Multiplica dos números
    def multiply_two_numbers(self, num1, num2):
        result = num1 * num2
        print(f"The result of multiplying {num1} by {num2} is: {result}")
        return result

    # Multiplica tres números
    def multiply_three_numbers(self, num1, num2, num3):
        result = num1 * num2 * num3
        print(f"The result of multiplying {num1}, {num2} and {num3} is:
{result}")
        return result

    # Divide dos números
    def divide_two_numbers(self, num1, num2):
        if num2 == 0:
            print("Error: Division by zero.")
            return 0
        result = num1 / num2
        print(f"The result of dividing {num1} by {num2} is: {result}")
        return result
```

```

# Divide tres números
def divide_three_numbers(self, *num1):
    if num2 == 0 or num3 == 0:
        print("Error: Division by zero.")
        return 0
    result = num1 / num2 / num3
    print(f"The result of dividing {num1} by {num2} and then by {num3}
is: {result}")
    return result

# Crear una instancia de la calculadora
calc = Calculator()

# Probar los métodos
calc.add_two_numbers(5, 10)
calc.add_three_numbers(5, 10, 15)
calc.subtract_two_numbers(20, 5)
calc.subtract_three_numbers(30, 10, 5)
calc.multiply_two_numbers(2, 3)
calc.multiply_three_numbers(2, 3, 4)
calc.divide_two_numbers(20, 5)
calc.divide_three_numbers(100, 5, 2)

```

AYUDAS:

- Generalizar métodos existentes (*)
- Eliminar duplicados

Ejemplo 3. Debugging.

```
import math

def calcular_area_triangulo(base, altura):
    area = (base + altura) / 2
    return area

def calcular_area_cuadrado(lado):
    area = lado * lado * 2
    return area

def calcular_area_hexagono(lado):
    area = 6 * (lado ** 2) * math.sin(math.radians(60))
    return area

def calcular_area_heptagono(lado):
    area = 7 * (lado ** 2) * math.sin(math.radians(51.43))
    return area

def calcular_area_circulo(radio):
    area = math.pi * radio * radio * 3
    return area

def obtener_valores_figura(figura):
    if figura == "triangulo":
        base = input("Introduce la base del triángulo: ")
        altura = input("Introduce la altura del triángulo: ")
        return float(base), float(altura)
    elif figura == "cuadrado":
        lado = input("Introduce el lado del cuadrado: ")
        return float(lado),
    elif figura == "hexagono":
        lado = input("Introduce el lado del hexágono: ")
        return float(lado),
    elif figura == "heptagono":
        lado = input("Introduce el lado del heptágono: ")
        return float(lado),
    elif figura == "circulo":
        radio = input("Introduce el radio del círculo: ")
        return float(radio),
```

```

else:
    print("Figura no válida")
    return None

def main():
    print("Selecciona una figura para calcular el área:")
    print("1. Triángulo")
    print("2. Cuadrado")
    print("3. Hexágono")
    print("4. Heptágono")
    print("5. Círculo")

    figura = input("Introduce el número de la figura: ")

    if figura == "1":
        figura = "triangulo"
    elif figura == "2":
        figura = "cuadrado"
    elif figura == "3":
        figura = "hexagono"
    elif figura == "4":
        figura = "heptagono"
    elif figura == "5":
        figura = "circulo"
    else:
        print("Opción no válida.")
        return

    valores = obtener_valores_figura(figura)

    if valores is None:
        return

    if figura == "triangulo":
        area = calcular_area_triangulo(*valores)
        print("El área del triángulo es: " + str(area)) # Resultado
esperado: 10.0 para base=4, altura=5
    elif figura == "cuadrado":
        area = calcular_area_cuadrado(*valores)
        print("El área del cuadrado es: " + str(area)) # Resultado
esperado: 25.0 para lado=5

```



```
elif figura == "hexagono":
    area = calcular_area_hexagono(*valores)
    print("El área del hexágono es: " + str(area)) # Resultado
esperado: 64.95 para lado=5
elif figura == "heptagono":
    area = calcular_area_heptagono(*valores)
    print("El área del heptágono es: " + str(area)) # Resultado
esperado: 90.85 para lado=5
elif figura == "circulo":
    area = calcular_area_circulo(*valores)
    print("El área del círculo es: " + str(area)) # Resultado
esperado: 78.53981633974483 para radio=5

main()
```

Ejemplo 4. Debugging 2.

```
class Producto:
    def __init__(self, nombre, precio, stock):
        self.nombre = nombre if nombre else "Producto sin nombre"
        self.precio = precio
        self.stock = stock

    def mostrar_informacion(self):
        if self.precio < 0 or self.stock < 0:
            print(f"Producto '{self.nombre}' tiene datos
incorrectos.")
        else:
            print(f"Producto: {self.nombre}, Precio: ${self.precio},
Stock: {self.stock}")

    def actualizar_stock(self, cantidad):
        self.stock += cantidad

    def aplicar_descuento(self, descuento):
        self.precio -= self.precio * (descuento / 100)

class TodoList:
    def __init__(self):
        self.tareas = []

    def agregar_tarea(self, producto):
        if not isinstance(producto, Producto):
            print("Error: Solo se pueden agregar objetos de tipo
Producto.")
        else:
            self.tareas.append(producto)

    def eliminar_tarea(self, nombre_producto):
        print(f"Simulando eliminación del producto
'{nombre_producto}'... Pero no se elimina realmente.")
        for tarea in self.tareas:
            if tarea.nombre == nombre_producto:
                tarea.stock = 0 # En lugar de eliminarlo, solo le
ponemos stock 0
```

```
        print(f"Producto '{nombre_producto}' encontrado,  
stock puesto a 0.")
```

```
def mostrar_tareas(self):  
    if not self.tareas:  
        print("No hay tareas en la lista.")  
    else:  
        for tarea in self.tareas:  
            tarea.mostrar_informacion()
```

```
# Ejemplo de uso de las clases con errores
```

```
todo_list = TodoList()
```

```
# Productos con datos incorrectos
```

```
producto1 = Producto("Laptop", 1000, 10) # Producto válido
```

```
producto2 = Producto("Smartphone", -500, 5) # Precio negativo  
(Error)
```

```
producto3 = Producto("Tablet", 300, -3) # Stock negativo (Error)
```

```
producto4 = Producto("", 200, 20) # Producto con nombre vacío  
(Error)
```

```
# Agregar productos a la lista de tareas
```

```
todo_list.agregar_tarea(producto1) # Producto válido
```

```
todo_list.agregar_tarea(producto2) # Error en precio
```

```
todo_list.agregar_tarea(producto3) # Error en stock
```

```
todo_list.agregar_tarea(producto4) # Nombre vacío
```

```
# Mostrar todas las tareas
```

```
todo_list.mostrar_tareas()
```

```
# Intentar eliminar un producto de la lista
```

```
todo_list.eliminar_tarea("Laptop") # Debería eliminar, pero no lo  
hará correctamente
```

```
todo_list.eliminar_tarea("Cargador") # Producto que no existe en la  
lista
```

```
todo_list.mostrar_tareas() # Mostrar después de intentar la  
eliminación
```

```
# Aplicar descuento erróneo a un producto
```

```
producto1.aplicar_descuento(150) # Descuento fuera de rango
```

```
producto2.aplicar_descuento(20) # Aplicar descuento correctamente
todo_list.mostrar_tareas() # Ver los cambios
```